Chapter 04: Constants and Strings

ENG3091

Khiem Nguyen

Contents

1 Constant variables **1**
- 1.1 Some values should never change . . . 1
- 1.2 Declaring a constant variable . . . 2
- 1.3 A `const` variable must be initialized . . . 2
- 1.4 Initializing from a variable . . . 2
- 1.5 Naming constants . . . 3
- 1.6 `const` value parameters . . . 3
- 1.7 `const` return values . . . 4
- 1.8 Why make variables constant? . . . 4
- 1.9 `constexpr`: constants known at compile time . . . 5

2 Literals **5**
- 2.1 Literals . . . 5
- 2.2 Magic numbers . . . 6

3 Strings **6**
- 3.1 `std::string`: text in C++ . . . 6
- 3.2 Your first strings . . . 7
- 3.3 Joining, indexing and comparing . . . 7
- 3.4 Reading a string: `>>` stops at a space . . . 8
- 3.5 `std::getline` reads the whole line . . . 8
- 3.6 Watch out: mixing `>>` and `std::getline` . . . 9
- 3.7 The fix: `std::ws` . . . 9
- 3.8 Strings and functions . . . 10

4 Wrapping up **10**
- 4.1 Summary . . . 10
- 4.2 Keep learning . . . 10

1 Constant variables

1.1 Some values should never change

So far, every variable could be changed at any time:

```
1 double x{42.0};
2 x = 24.0;        // fine: x is meant to change
3
4 double gravity{9.81};
```

```
gravity = 1.89;  // allowed... but gravity shouldn't change!

double pi{3.1415926535};
pi = 3.0;        // some engineers do see pi as 3.0, but we don't want that!
```

A **constant** is a value that can't change while the program runs. C++ has two kinds:

- **named constants**, which have a name (this section);
- **literal constants**, or just **literals**, written straight into the code, like `9.81` (next section).

1.2 Declaring a constant variable

Put the keyword `const` (the **const qualifier**) next to the type:

```
const double gravity{9.81};    // the usual style
int const sides_in_square{4};  // also legal, but less common
```

- `const` can go before or after the type. Before is more common: think "a green ball", not "a ball green".
- `const` is part of the type: the type of `gravity` is `const double`.

1.3 A `const` variable must be initialized

You can't assign to a `const` variable, so it must get its value when it's defined:

Listing 1 `const_must_be_initialized.cpp`

```
int main() {
    const double gravity; // error: no initializer
    gravity = 9.81;       // error: can't assign to a const
    return 0;
}
```

```
error: uninitialized 'const gravity' [-fpermissive]
error: assignment of read-only variable 'gravity'
```

1.4 Initializing from a variable

A `const` variable can be initialized with any value, even one that's only known when the program runs:

Listing 2 `initialize_const_variables.cpp`

```cpp
#include <iostream>

int main() {
    std::cout << "Enter your age: ";
    int age{};
    std::cin >> age;

    const int const_age{age}; // fine: initialized from a variable

    age = 5;       // fine: age is not const
    const_age = 6; // error: const_age is const
    return 0;
}
```

```
error: assignment of read-only variable 'const_age'
```

1.5 Naming constants

You'll meet several conventions:

- `EARTH_GRAVITY`: upper case with underscores, popular with programmers coming from C;
- `kEarthGravity`: a "k" prefix, common in some C++ style guides;
- `earth_gravity`: just like any other variable.

My take: a `const` variable behaves like a normal variable except that it can't be assigned to, so it doesn't need a special name. Your IDE will tell you it's `const` anyway. In this course we use `snake_case` for constants too.

1.6 `const` value parameters

Listing 3 `const_func_parameters.cpp`

```cpp
#include <iostream>

void print_double(const int x) { // x is read-only
    std::cout << x * 2 << '\n';
}

int main() {
    print_double(21);
    return 0;
}
```

```
42
```

- A `const` **value parameter** can't be changed inside the function.
- But it's only a copy (pass by value, Chapter 02): whatever the function does to it, nobody outside can tell. So the `const` adds clutter and protects nothing.

1.7 `const` return values

Listing 4 `const_return_value.cpp`

```cpp
#include <iostream>

const double get_value() { // const here does nothing
    return 42.0;
}

int main() {
    std::cout << get_value() << '\n';
    return 0;
}
```

```
warning: type qualifiers
ignored on function
return type
```

A `const` return value of a fundamental type is simply ignored, and `-Wextra` says so.

> Best practice
>
> Don't use `const` on value parameters or on values returned by value. (`const` *references* are a different story: Chapter 07.)

1.8 Why make variables constant?

If a variable can be made constant, it generally should be.

- **No accidents.** Nobody, including you, can change it by mistake.
- **Faster programs.** When the compiler knows a value won't change, it can optimize more.
- **Simpler code.** When you're reading or debugging, a `const` variable is one less moving part: you never have to ask whether, where or to what it changed.

Look at well-written open-source C++ libraries on GitHub: you'll see `const` almost everywhere.

1.9 `constexpr`: constants known at compile time

Listing 5 `constexpr_example.cpp`

```cpp
#include <iostream>
#include <numbers> // C++20: std::numbers::pi

int main() {
    constexpr double pi{std::numbers::pi}; // compile time

    std::cout << "Radius: ";
    double radius{};
    std::cin >> radius;

    const double area{pi * radius * radius}; // run time
    std::cout << "Area: " << area << '\n';
    return 0;
}
```

```
$ ./constexpr_example
Radius: 2
Area: 12.5664
```

> **Best practice**
>
> `constexpr` for values known at compile time, `const` for the rest.

- A `constexpr` variable needs a value the compiler can work out **while compiling**, like `9.81` or `std::numbers::pi` (from `<numbers>`, C++20: don't type `pi` yourself!).
- `area` depends on user input, so it's only known when the program **runs**: it can be `const` but not `constexpr`.

2 Literals

2.1 Literals

A **literal** is a value written directly in the code. Like a variable, every literal has a type, and C++ works it out from how the literal is written:

Literal	Type	Literal	Type
`42`	`int`	`'A'`	`char` (single quotes)
`42u`	`unsigned int`	`"Hello"`	text (double quotes)
`3.14`	`double`	`true`, `false`	`bool`
`3.14f`	`float`	`1.5e3`	`double` (1500.0)

> **Coming from Python**
>
> In Python, `'A'` and `"A"` mean the same thing. In C++ they don't: single quotes make one `char`, double quotes make text.

2.2 Magic numbers

Remember `if (height >= 140)` from Chapter 03? What is `140`: a height, a weight, an age? A literal with no explanation is a **magic number**, and if the rule changes, you have to hunt down every copy.

Listing 6 `magic_numbers.cpp`

```cpp
#include <iostream>

int main() {
    constexpr int min_ride_height_cm{140}; // named, not magic

    std::cout << "Enter your height (in cm): ";
    int height{};
    std::cin >> height;

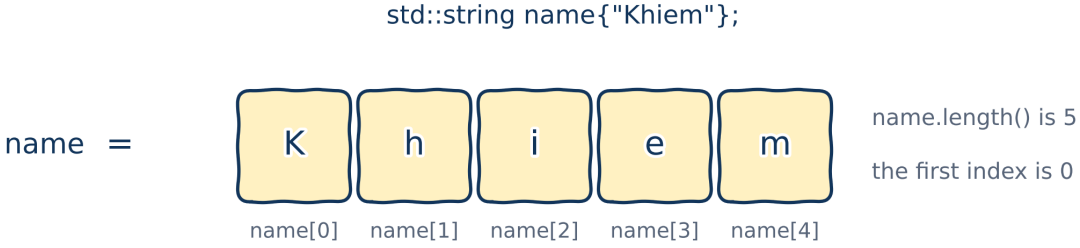
    if (height >= min_ride_height_cm) {
        std::cout << "You are tall enough to ride.\n";
    } else {
        std::cout << "Sorry, too short to ride.\n";
    }
    return 0;
}
```

```
$ ./magic_numbers
Enter your height (in cm): 120
Sorry, too short to ride.
```

> **Best practice**
>
> Give magic numbers a name with `constexpr`.

3 Strings

3.1 `std::string`: text in C++



- A `std::string` holds a piece of text: a sequence of characters, numbered from **0**.
- It lives in the standard library, so `#include <string>`.

3.2 Your first strings

Listing 7 `string_basics.cpp`

```cpp
#include <iostream>
#include <string> // for std::string

int main() {
    std::string name{"Khiem"};
    std::cout << "Hello, " << name << '\n';

    name = "Ada Lovelace"; // it can change, and grow
    std::cout << "Hello, " << name << '\n';
    std::cout << name << " has " << name.length() << " characters\n";
    return 0;
}
```

```
Hello, Khiem
Hello, Ada Lovelace
Ada Lovelace has 12 characters
```

- A string can change, and it grows or shrinks as needed: no need to say how long it is.
- `name.length()` tells you how many characters it holds. (It counts the space too.)

3.3 Joining, indexing and comparing

Listing 8 `string_operations.cpp`

```cpp
#include <iostream>
#include <string>

int main() {
    std::string first{"Khiem"};
    std::string last{"Nguyen"};
    std::string full{first + " " + last}; // join with +

    std::cout << full << '\n';
    std::cout << "First letter: " << full[0] << '\n'; // from 0

    if (first == "Khiem") { // compare with ==
        std::cout << "Hello, lecturer!\n";
    }
    return 0;
}
```

```
Khiem Nguyen
First letter: K
Hello, lecturer!
```

> **Coming from Python**
>
> `+` joins and `==` compares, as in Python; `len(s)` becomes `s.length()`. But a C++ string can be changed in place (Python strings can't), and `s[-1]` does **not** give the last character: there are no negative indices.

3.4 Reading a string: >> stops at a space

Listing 9 `string_input_cin.cpp`

```cpp
#include <iostream>
#include <string>

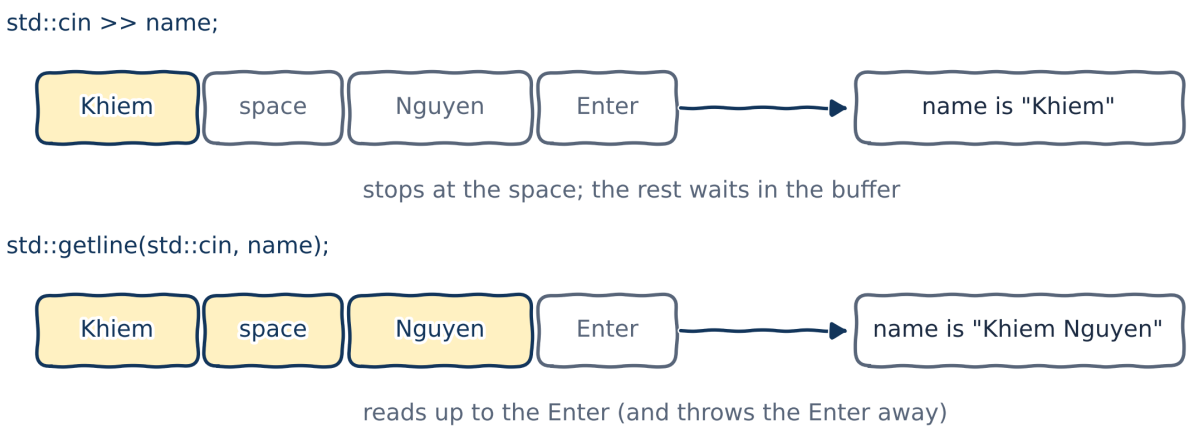
int main() {
    std::cout << "Enter your full name: ";
    std::string name{};
    std::cin >> name; // stops at the first space!

    std::cout << "Hello, " << name << '\n';
    return 0;
}
```

```
$ ./string_input_cin
Enter your full name: Khiem Nguyen
Hello, Khiem
```

Where did "Nguyen" go? `>>` reads one *word*: it stops at the first space, and the rest stays in the input buffer (remember Chapter 01?).

3.5 `std::getline` reads the whole line



To read text that may contain spaces, use `std::getline(std::cin, name)`. It reads everything up to the Enter key.

3.6 Watch out: mixing >> and `std::getline`

Listing 10 `string_input_getline_bug.cpp`

```cpp
#include <iostream>
#include <string>

int main() {
    std::cout << "Enter your age: ";
    int age{};
    std::cin >> age;

    std::cout << "Enter your full name: ";
    std::string name{};
    std::getline(std::cin, name); // gets the leftover Enter

    std::cout << "Hello, " << name << ", age " << age << '\n';
    return 0;
}
```

```
$ ./string_input_getline_bug
Enter your age: 35
Enter your full name: Hello, , age 35
```

The program didn't even wait for the name! `>>` read `35` but left the **Enter** in the buffer, and `std::getline` happily read that empty line.

3.7 The fix: `std::ws`

Listing 11 `string_input_getline.cpp`

```cpp
#include <iostream>
#include <string>

int main() {
    std::cout << "Enter your age: ";
    int age{};
    std::cin >> age; // leaves Enter in the buffer

    std::cout << "Enter your full name: ";
    std::string name{};
    std::getline(std::cin >> std::ws, name); // skip it, then read a line

    std::cout << "Hello, " << name << ", age " << age << '\n';
    return 0;
}
```

```
$ ./string_input_getline
Enter your age: 35
Enter your full name: Khiem Nguyen
Hello, Khiem Nguyen, age 35
```

> **Best practice**
>
> `std::ws` skips leftover spaces and Enters, so this works whatever came before.

3.8 Strings and functions

```cpp
void print_greeting(std::string name) { // pass by value: a copy
    std::cout << "Hello, " << name << '\n';
}
```

- Passing a `std::string` by value works, but it **copies** every character. For a name that's nothing; for a whole book it's slow.
- In Chapter 07 we'll learn to pass strings without copying them, using **references**. C++17 also offers `std::string_view`, a cheap read-only view of a string.
- Until then, passing strings by value is fine.

4 Wrapping up

4.1 Summary

- `const` makes a variable read-only. It must be initialized, and **if a variable can be `const`, it should be**.
- `constexpr` is for values known at compile time; `const` for the rest. Get `pi` from `<numbers>`.
- Don't put `const` on value parameters or on return values.
- Every literal has a type: `'A'` is a `char`, `"A"` is text. Replace magic numbers with named `constexpr` constants.
- `std::string` holds text: `+` joins, `==` compares, `s[0]` is the first character, `s.length()` counts them.
- `>>` reads one word; `std::getline(std::cin >> std::ws, s)` reads a whole line.

4.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Literals, including suffixes like `u` and `f`
2. Numeral systems: binary, hexadecimal and octal
3. Introduction to `std::string` and `std::string_view`
4. Constexpr variables

Next up: Section 2, C++ Intermediate, starting with function overloading.